

ENGINEERING REPORT

Meter-Triggered Latching Control System for a 500 A Contactor

Design, Development, Iterative Testing & Deployment

Prepared for: **Thal Engineering**

Prepared by: **Humayun Khan**
R&D Engineer (Freelance)

Date: 13 July 2026

Report Ref: THAL-CTR-2026-07

Repository: [Latching-Relay-HV-System](#)

Executive Summary

This report documents the design and deployment of a controller that reads a utility meter's connect/disconnect (ON/OFF) signalling contacts and uses them to latch a 500 A contactor — energized on connect, de-energized on disconnect — reliably, in an electrically noisy environment (nearby ACs, motors, and the contactor's own switching transients).

The deployed solution uses two PC817 optocouplers to isolate and clean the meter's signal, an ESP32-C3 to detect the resulting sustained-LOW signal and hold a set/reset latch, and a Fetek SSR-40DA to switch the contactor coil with no mechanical contacts. Three isolated supply domains are used throughout.

A parallel evaluation of commercial latching relays found no suitable drop-in unit (Section 6). The opto-isolated, microcontroller-latched, SSR-driven design was adopted as final; three units are built, deployed, and verified under live load.

1. Objective

The site meter operates its own motorized breaker on a remote connect/disconnect command, but the actual load is switched by a separate 500 A contactor that the meter cannot drive directly. The objective was a control circuit that: reads the meter's ON/OFF signalling outputs without loading or damaging them; converts its momentary pulses into a maintained, latched contactor state; survives the site's electrical noise; and is safe, documented, and reproducible.

2. System Overview

Three isolated domains, connected only through the optocouplers and the SSR: the field side (meter contacts + dedicated 5 V supply), the logic side (ESP32-C3 + its own 5 V supply), and the load side (500 A contactor coil + its coil supply, switched by the SSR). The meter's ON/OFF contacts each drive a PC817; the phototransistor output is read on a GPIO (active-low); firmware applies a set/reset latch and drives GPIO10, which switches the SSR into the contactor coil circuit.

3. Development Iterations

The final design was reached through six iterations, each driven by an observed failure of the previous step:

Iter.	Activity and outcome
I	Voltage-based sensing (ZMPT101B). Attempted to infer meter state from load-side AC voltage. Set aside in favour of reading the meter's own dedicated ON/OFF signalling contacts directly.
II	Direct contact sensing — inconsistent capture. Wired straight to GPIOs; one channel captured reliably, the other missed or read as permanently closed. Showed the assumed edge-detection method did not match the meter's real signal.
III	Characterization of the true signal. A high-speed diagnostic sniffer revealed each command is a dense burst of thousands of very short (5–113 μ s) pulses over several seconds, not a single clean closure (full data in Section 4).
IV	Move to opto-isolated sensing. PC817 optocouplers isolate the meter's contacts and, importantly, integrate the pulse burst into one clean, sustained logic transition — removing any direct ground connection to the meter.
V	Electromechanical relay output — welding and EMI. An electromechanical relay switching the real 500 A contactor coil worked on the bench but later welded shut (unsuppressed inductive arcing) and injected EMI that hung the microcontroller.
VI	Solid-state output (Fetek SSR) + corrected polarity. Replaced the relay with a Fetek SSR-40DA (no contacts to weld, zero-cross switching) and finalized sense-channel polarity/grounding (meter COM to opto-side ground, ON/OFF terminals into the opto LEDs). This is the deployed configuration.

4. Signal Characterization

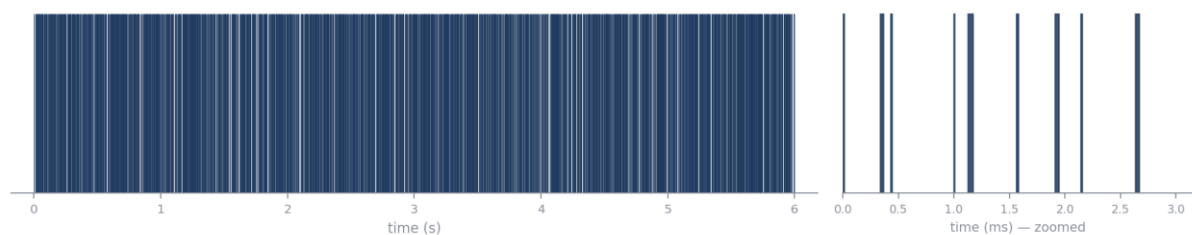
The meter's real signal, and how it changes across the opto-isolation stage, is the central technical finding of this project.

4.1 Raw signal (measured directly on the dry contacts)

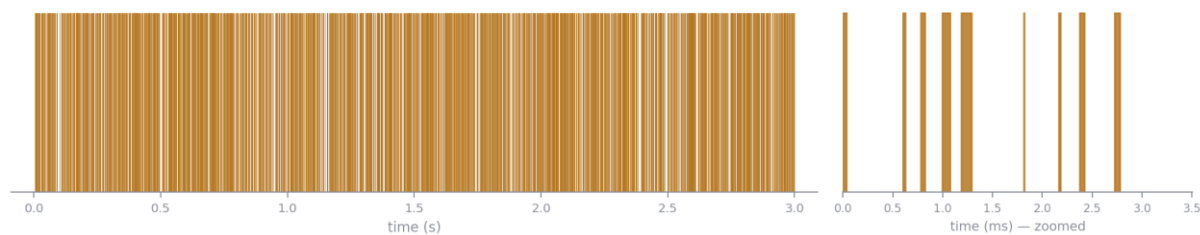
Raw (bare-wire) signal	CONNECT	DISCONNECT
Pulse width	5–43 μ s	5–113 μ s
Closes / command	$\approx 4,000$ / ≈ 6 s	$\approx 3,000$ / ≈ 3 s
Idle	≈ 0 closes/s	≈ 0 closes/s

Raw meter signal — measured directly on the dry contacts

CONNECT (ON) — $\approx 4,000$ pulses over ≈ 6 s



DISCONNECT (OFF) — $\approx 3,000$ pulses over ≈ 3 s



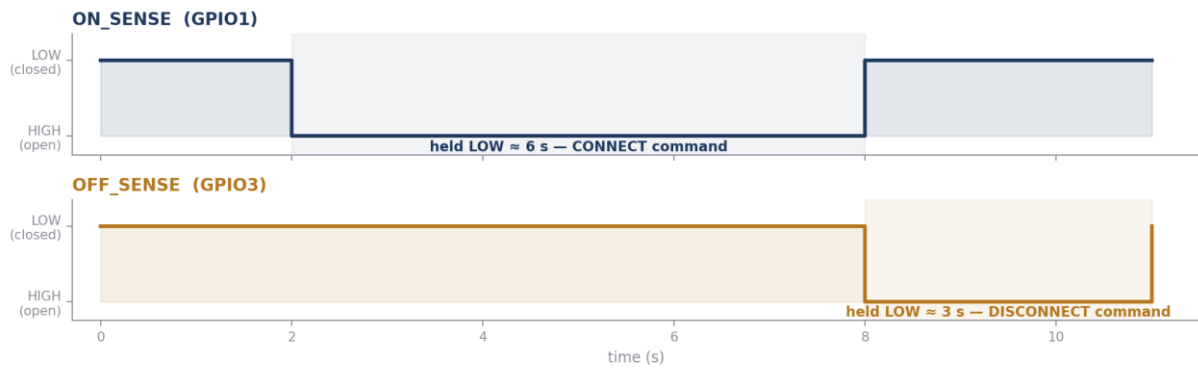
Idle periods between commands are silent (≈ 0 closes/s on both channels). Pulse count is illustrative of density, not an exact reproduction.

Figure 1 — Raw signal: a dense burst of very short pulses per command (illustrative density; zoomed insets show individual pulse widths).

4.2 Signal after opto-isolation (as read by the ESP32-C3)

The PC817's finite response time integrates the burst into one clean, sustained closure:

Post-optocoupler signal	CONNECT	DISCONNECT
Held LOW for	≈ 6 s	≈ 3 s
Consistency	consistent	3000–3010 ms
Mutual exclusivity	ON low $\Rightarrow$ OFF open	OFF low $\Rightarrow$ ON open

Signal after opto-isolation — sustained level, as read by the ESP32-C3 GPIO

The two channels are mutually exclusive (never LOW together). Detection rule: a line held LOW continuously for ≥ 800 ms (THRESHOLD_MS) is accepted as that command — comfortably above any noise spike, comfortably below the ≈ 3 – 6 s real signal.

Figure 2 — Sustained-level signal as seen by the microcontroller after opto-isolation.

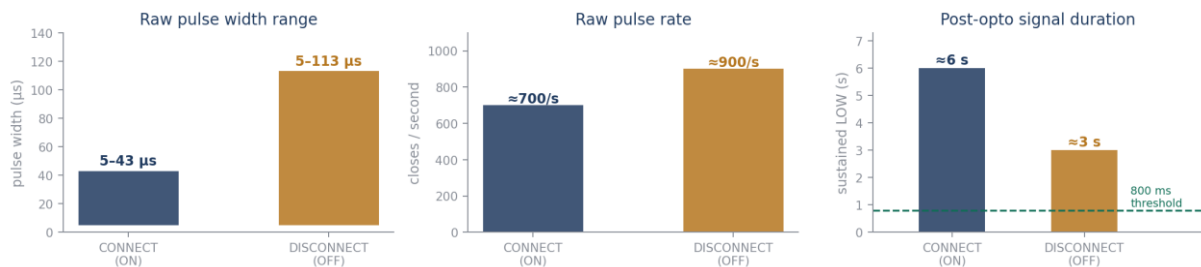
Signal metrics — CONNECT vs DISCONNECT

Figure 3 — Pulse width, rate, and post-opto duration, CONNECT vs. DISCONNECT.

CONNECT (≈ 6 s) is longer than DISCONNECT (≈ 3 s) because closing/spring-charging the breaker takes longer than releasing it. A handful of "missed" disconnects during testing were meter-side non-actuations, not lost signals — confirmed against the dashboard.

Detection rule adopted: a line held LOW continuously for ≥ 800 ms is accepted as that command — comfortably above any noise glitch and comfortably below the ≈ 3 s real signal.

5. Final Circuit and Protective Measures

5.1 Isolation

- Three independent supply domains (field / logic / load); optocouplers and the SSR are the only crossings.
- PC817s isolate the meter's field wiring from the microcontroller's logic ground.
- SSR isolates the control output from the 500 A contactor coil circuit.

5.2 Output stage

- Fotek SSR-40DA (zero-cross, 4–32 VDC control) replaces the electromechanical relay — no contacts to weld.
- RC snubber across the contactor coil limits the inductive turn-off transient.

5.3 Firmware reliability

- Polling only (no interrupts) — immune to the interrupt-storm hang seen in Iteration V.
- Non-blocking loop; hardware watchdog auto-resets on any stall (≤ 8 s); flash-persisted state restores automatically after a reset; reset-reason reporting distinguishes brownout from EMI on every boot.

5.4 Sensing logic

- Sustained-duration detection (≥ 800 ms), matched to the measured post-opto signal; idempotent set/reset latch; ON/OFF confirmed mutually exclusive.

6. Latching / Standard Relay Feasibility Study

Commercial latching (impulse) relays were evaluated as a non-microcontroller alternative:

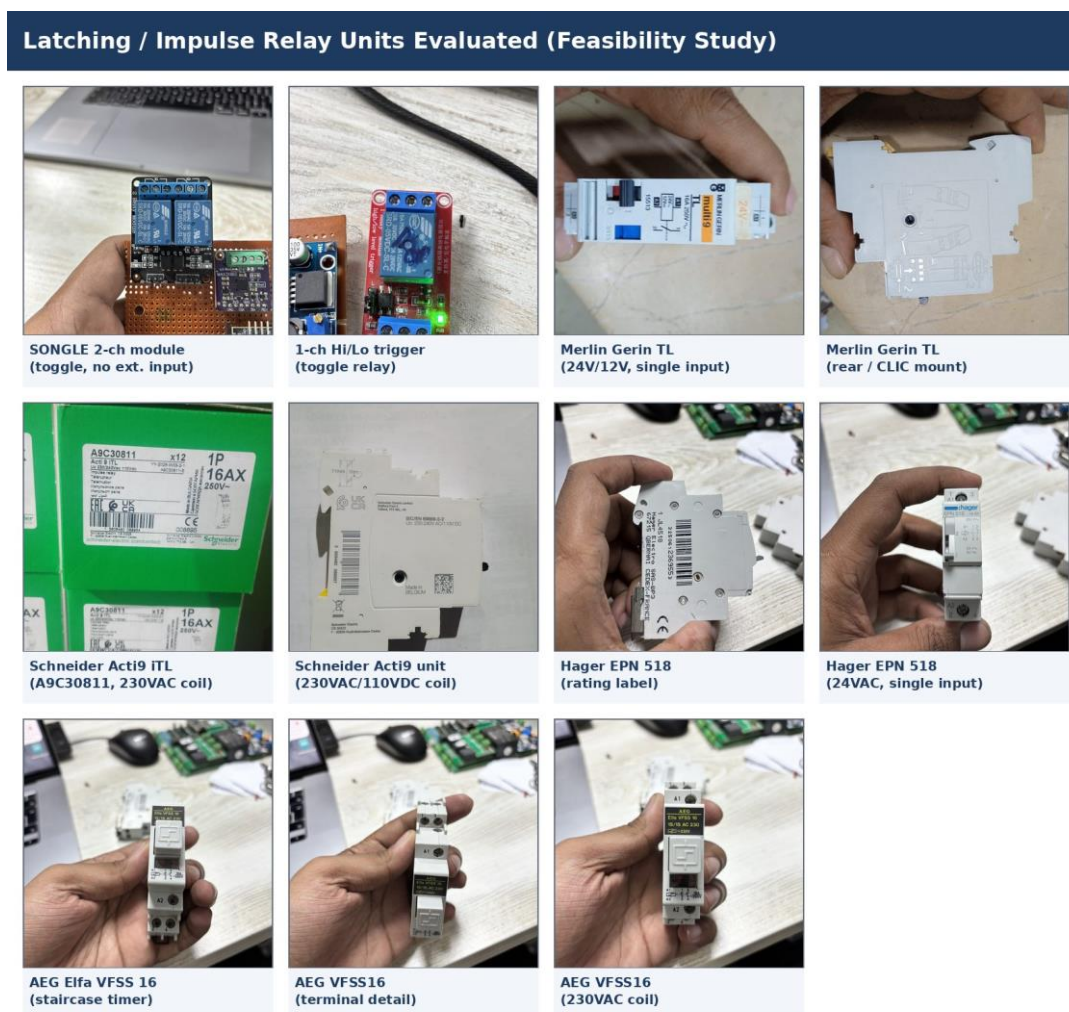


Figure 4 — Latching/impulse relay units evaluated during the feasibility study.

Device evaluated	Reason not suitable
Schneider/Merlin Gerin TL, iTL (A9C30811)	Single-input toggle — cannot distinguish connect from disconnect. 230 VAC/110 VDC coil, not 12 V/5 V.
Hager EPN 518	Single-input toggle; 24 VAC coil; two contacts but one control input.
ABB E290-16-10/230	Single-input toggle + manual override handle; 230 VAC coil.
AEG Elfa VFSS 16	A staircase/time-delay relay, not a latch — reverts on its own; 230 VAC coil.

Root causes: (1) the job needs two independent control inputs — only a relay with dedicated central ON/OFF terminals (e.g. Schneider iTLc) has this, not the plain toggle units evaluated; (2) most stock units use 230 VAC coils against the 12 V/5 V available; (3) the meter's outputs are low-current opto-isolated signals, while a relay coil draws 100–300 mA — any relay route still needs the same opto/transistor buffer as the deployed design, without its logging or firmware safeguards. The opto + ESP32-C3 + SSR architecture was therefore adopted as final.

7. System Range, Capabilities & Limitations

7.1 Capabilities

- Scales to any contactor coil current rating — the SSR switches only the coil, not the load path; verified at 500 A.
- ~800 ms detection response against a ≥ 3 s real signal — wide margin.
- Self-recovering: watchdog auto-reset (≤ 8 s) plus flash-restored state — no manual intervention after a transient fault.
- Field-diagnosable via boot-time reset-reason reporting (brownout vs. EMI).
- Serial override (1/0/t) for commissioning without the meter.
- Fully isolated three-ground design — safe alongside heavy inductive loads (motors, ACs).

7.2 Limitations

- Detection thresholds are tuned to this meter's measured pulse/duration profile; a different meter or a firmware change on the meter side may require re-tuning.
- The system mirrors the meter's commanded state; it cannot force the meter's own breaker to actuate (occasional missed first-presses are meter-side).
- SSR-40DA's rated input minimum (4 V) is technically above the ESP's 3.3 V logic-high; reliable in the deployed unit, but a 5–12 V driver stage is recommended for full margin in future builds.
- No auxiliary feedback wired from the contactor itself — the system confirms a commanded state, not physical contactor position.
- This is a control follower only; it provides no load metering or overcurrent protection — upstream breakers/fuses remain the site's primary protection.

8. Conclusion

Through six iterations, the meter's momentary connect/disconnect signalling was correctly characterized as a dense microsecond-pulse burst that opto-isolation cleanly integrates into a sustained, several-second signal — the basis of a simple, robust detection rule. An electromechanical relay output stage failed under the real 500 A contactor coil (welding, EMI); replacing it with a Fotek SSR-40DA, together with three-way isolation and firmware safeguards (polling, watchdog, state persistence, reset diagnostics), resolved both issues. A parallel evaluation of commercial latching relays found no suitable drop-in unit. The opto-isolated, microcontroller-latched, SSR-driven architecture is the deployed solution, verified across three units under live load.

Appendix A — Bill of Materials

Component	Key specification	Role
ESP32-C3 SuperMini	RISC-V SoC, Wi-Fi/BLE, 3.3 V logic	Control logic, latching
PC817 optocoupler (×2)	4-pin DIP, 5 kV isolation	Isolated signal sensing
Fotek SSR-40DA	In 4–32 VDC, out 24–380 VAC, 40 A	Contactor coil switching
HLK-5M05 (×2)	85–264 VAC in, 5 V/1 A out	Independent field & logic supplies
RC snubber	0.1 μ F X2 + $\approx$ 100 Ω	Coil transient suppression
500 A contactor	AC coil (existing, deployed)	Load switching

Appendix B — Repository & Related Work

github.com/HumayunNaveedKhan/Latching-Relay-HV-System

Appendix C — Finalized Code and Circuit Diagram

```

/*
 * ESP32-C3 — Hexing meter -> contactor controller (FINAL, opto + SSR)
 * -----
 * Signal (measured, through the optocouplers):
 * CONNECT (ON) -> GPIO1 held LOW ~6 s (sometimes 3 s + blip + 3 s)
 * DISCONNECT (OFF) -> GPIO3 held LOW ~3 s
 * Mutually exclusive. Real closures are 3000 ms+; noise can't hold an
 * isolated opto LOW at all. So: a line LOW past THRESHOLD_MS = that command.
 *
 * WIRING
 * ON contact -> opto -> GPIO1 (active-low: closure = LOW, INPUT_PULLUP)
 * OFF contact -> opto -> GPIO3
 * SSR input + -> GPIO10, SSR input - -> ESP GND (active-high: HIGH = ON)
 * SSR output -> in series with the 220V contactor coil (its own HiLink)
 * Opto LED side on a SEPARATE supply; its GND ties to meter COM only.
 *
 * LATCH: ON command -> SSR ON (held) ; OFF command -> SSR OFF (held).
 * OVERRIDE (Serial @115200): 1 = ON, 0 = OFF, t = toggle.
 * ALIVE: no interrupts (no storms), no blocking calls, watchdog auto-recovers
 * any hang, and the last state is restored on boot so a reset is seamless.
 */

#include "esp_task_wdt.h"
#include <Preferences.h>

#define PIN_ON 1
#define PIN_OFF 3
#define PIN_SSR 10 // ACTIVE-HIGH SSR: HIGH = ON

#define THRESHOLD_MS 800 // sustained LOW to accept a command (real >= 3000 ms)
#define WDT_TIMEOUT 8 // seconds before a hung loop is auto-reset
#define RESTORE_STATE true // restore last state on boot

Preferences prefs;
bool relayState = false; // true = SSR ON (contactor connected)

uint32_t onLowStart = 0, offLowStart = 0;

```

```

bool    onFired = false, offFired = false;
int     onPrev = HIGH, offPrev = HIGH;

void driveOutput() { digitalWrite(PIN_SSR, relayState ? HIGH : LOW); } // active-high

void setRelay(bool on) {
  if (on == relayState) { driveOutput(); return; } // idempotent
  relayState = on;
  driveOutput();
  prefs.putBool("state", relayState); // survive resets
  Serial.printf("%lu ms -> %s\n", millis(),
    on ? "CONNECT (SSR ON)" : "DISCONNECT (SSR OFF)");
}

void printResetReason() {
  const char* s;
  switch (esp_reset_reason()) {
    case ESP_RST_POWERON: s = "power-on"; break;
    case ESP_RST_SW: s = "software"; break;
    case ESP_RST_PANIC: s = "PANIC / crash (EMI?)"; break;
    case ESP_RST_INT_WDT: s = "INTERRUPT wdt (EMI?)"; break;
    case ESP_RST_TASK_WDT: s = "TASK wdt / hang (EMI?)"; break;
    case ESP_RST_WDT: s = "watchdog"; break;
    case ESP_RST_BROWNOUT: s = "BROWNOUT / supply dip"; break;
    default: s = "other"; break;
  }
  Serial.printf("Reset reason: %s\n", s);
}

void setup() {
  Serial.begin(115200);
  delay(150);

  pinMode(PIN_SSR, OUTPUT);
  digitalWrite(PIN_SSR, LOW); // OFF first (active-high SSR)

  printResetReason();

  prefs.begin("relay", false);
  relayState = RESTORE_STATE ? prefs.getBool("state", false) : false;
  delay(200);
  driveOutput();
  Serial.printf("Restored: %s\n", relayState ? "ON" : "OFF");

  pinMode(PIN_ON, INPUT_PULLUP);
  pinMode(PIN_OFF, INPUT_PULLUP);

#ifdef ESP_ARDUINO_VERSION_MAJOR >= 3
  esp_task_wdt_config_t wdt = { .timeout_ms = WDT_TIMEOUT * 1000,
    .idle_core_mask = 0, .trigger_panic = true };
  esp_task_wdt_reconfigure(&wdt);
#else
  esp_task_wdt_init(WDT_TIMEOUT, true);
#endif
  esp_task_wdt_add(NULL);

  Serial.println(F("Ready. 1=ON 0=OFF t=toggle"));
}

void loop() {
  esp_task_wdt_reset(); // keep-alive; never blocks below

```

```
uint32_t now = millis();
```

```
int on = digitalRead(PIN_ON);
```

```
int off = digitalRead(PIN_OFF);
```

```
// CONNECT: GPIO1 held LOW past the threshold
```

```
if (on == LOW) {
```

```
    if (onPrev == HIGH) { onLowStart = now; onFired = false; }
```

```
    if (!onFired && (now - onLowStart) >= THRESHOLD_MS) { onFired = true; setRelay(true); }
```

```
} else onFired = false;
```

```
onPrev = on;
```

```
// DISCONNECT: GPIO3 held LOW past the threshold
```

```
if (off == LOW) {
```

```
    if (offPrev == HIGH) { offLowStart = now; offFired = false; }
```

```
    if (!offFired && (now - offLowStart) >= THRESHOLD_MS) { offFired = true; setRelay(false); }
```

```
} else offFired = false;
```

```
offPrev = off;
```

```
// serial override
```

```
if (Serial.available()) {
```

```
    char c = Serial.read();
```

```
    if (c == '1' || c == 'o' || c == 'O') setRelay(true);
```

```
    else if (c == '0' || c == 'f' || c == 'F') setRelay(false);
```

```
    else if (c == 't' || c == 'T') setRelay(!relayState);
```

```
}
```

```
}
```

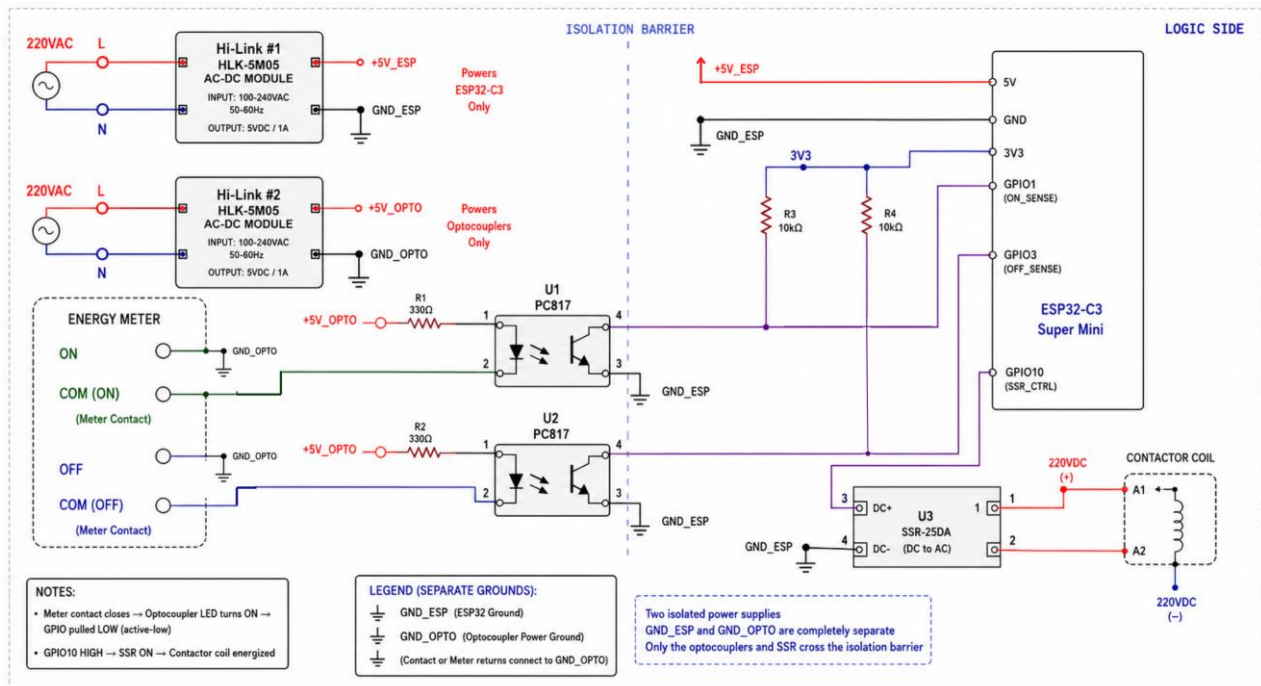


Figure 5 — Final Circuit Diagram